UNIVERSITY OF THE WITWATERSRAND, JOHANNESBURG
School of Computer Science & Applied Mathematics

Advanced Analysis of Algorithms
Lab 2: Uninformed to Informed Search

1 Introduction

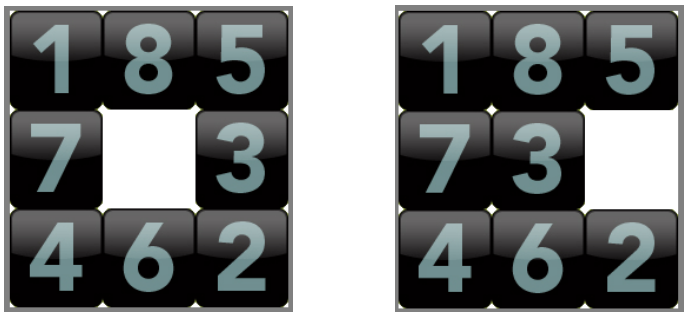
In this lab, we'll be extending your implementation of the breadth first search by changing the cost function and adding in heuristic functions. Please note that this lab is **not for marks**. While there are submission links on Moodle and some of the submissions have mark indications, this is purely for your own information and to help you align on what to expect for the lab test. While the lab is not compulsory, I highly recommend that you complete it well in advance of the lab test to ensure you are prepared.

2 8-Puzzle

The 8-Puzzle is a game in which a player is required to slide tiles around on a 3×3 grid in order to reach a goal configuration. If you have never played before, I recommend trying it out here: `http://www.artbylogic.com/puzzles/numSlider/numberShuffle.htm?rows=3&cols=3&sqr=1`.

There is one empty space that an adjacent tile can be slid into. An alternative (and simpler) view is that we are simply moving the blank square around the grid. Note that not all actions can be used at all times. For example, the blank tile cannot be shifted up if it is already on the top row!

The transition below illustrates the blank tile being moved to the `right`.



(a) (b)

Figure 1: Starting in the configuration (a), the agent executes the action `right`. This moves the blank tile to the right, exchanging its position with 3, as seen in (b).

Submission 1: Uniform-cost search (10 marks)

Lets begin by modifying our code for the BFS to implement a UCS. First, modify the problem so that all actions have cost 1, except for `UP`, which now has a cost of 5 (do not remove the ability to go back to uniform costs later on). Then, write a Python script that accepts an initial and goal state, and returns the cost of the shortest path between the two according to UCS.

Input

Input consists of two lines. The first line is a representation of the initial state, and the second line is the goal state.

Output

Output the cost of the optimal plan from the start to the goal state.

Example Input-Output Pairs

Sample Input #1	Sample Input #2	Sample Input #3
78651#432 12345678#	1857#3462 1#5783462	1857#3462 78651432#
Sample Output #1	**Sample Output #2**	**Sample Output #3**
49	2	36

Submission 2: Greedy Best-First Search (20 marks)

Now lets move away from uninformed searches and begin to consider informed searches. To begin with we will drop the cost counting entirely and just use a heuristic function. Return the problem to the setting where all actions have cost 1 (including for `UP`). Then, write a Python script that accepts an initial and goal state, and returns the cost of the path between the two states (path cost is still counted in terms of number of moves to the white square to reach the goal state). Note that, because greedy-BFS is not guaranteed to final the optimal path you may have some variance in the path length between runs.

The heuristic function you should use is the sum of the manhattan distance for each tile from its final destination in the 3×3 grid. Reminder that the manhattan distance in $\mathbb{R}^N$ is:

$$d(x, y) = \sum_{i=0}^{N} |x_i - y_i|$$

Input

Input consists of two lines. The first line is a representation of the initial state, and the second line is the goal state.

Output

Output the cost of the found plan from the start to the goal state.

Example Input-Output Pairs

Sample Input #1	Sample Input #2	Sample Input #3
`5#1742638` `524316#87`	`1738546#2` `13648572#`	`3547#1628` `71246#835`
Sample Output #1	**Sample Output #2**	**Sample Output #3**
`31`	`49`	`63`

Submission 3: A* search (20 marks)

Now let's combine our implementations of BFS and greedy-BFS to implement A*. Once again, use the version of the problem where all actions have cost 1 and the manhattan distance as your heuristic function. Then write a Python script that accepts an initial and goal state, and returns the cost of the shortest path between the two according to A*.

Input

Input consists of two lines. The first line is a representation of the initial state, and the second line is the goal state.

Output

Output the cost of the optimal plan from the start to the goal state.

Example Input-Output Pairs

Sample Input #1	Sample Input #2	Sample Input #3
5#1742638 524316#87	1738546#2 13648572#	3547#1628 71246#835
Sample Output #1	**Sample Output #2**	**Sample Output #3**
19	25	21

Submission 4: Experimenting with Our Algorithms

For this lab we will be experimenting with the time complexity of the informed search algorithms with different heuristic functions. Similar to Lab 1 we need to control the complexity of the problem we ask the algorithms to solve. Use your program that generates game boards which require k moves to solve. Once the desired number of moves have been made, pass the resulting game state into your greedy-BFS and A* functions and set the goal state to be $12345678\#$ (the solved puzzle).

Repeat this process for variable solution depths k and plot this relationship with the time complexity of your algorithms. Measure time complexity using the wall-clock time from the start to the end of the search algorithm. For a given value of k repeat the algorithm 5 times and plot both the mean and 2 standard deviations for both complexity measures. Finally, compare these results to what happens when the euclidean distance is used as the heuristic function instead. Reminder that the euclidean distance in $\mathbb{R}^N$ is:

$$d(x, y) = \sqrt{\sum_{i=0}^{N} (x_i - y_i)^2}$$

Plot the complexity measures over different values of k with variance indicated and compare the heuristic functions.

Input

There is no input as you will begin with $12345678\#$ as the game state and choose a value of k to run the experiment.

Things to Consider

The types of things you should take note of and that I may ask (I may also show you a plot like the one I have asked you to create and ask questions from it):

1. The optimality of the greedy-BFS and A* algorithms. (10 marks)
2. The time complexity of the two algorithms using the manhattan distance as the heuristic function. (10 Marks)
3. The effect of changing the heuristic function on the optimality of the algorithms. (10 Marks)
4. The effect of changing the heuristic function on the time complexity of the algorithms. (10 marks)